

Regression

Regression is a statistical and machine learning technique used to predict continuous numerical values and understand the relationships between variables.

Regression tries to determine how a dependent variable and one or more other (independent) variables relate to each other. It's a statistical method.

ex:- house price, loan approval

linear regression simple model for predict continuous values it find the relationship between to input features it is use for predictions find the best fitting

types

simple linear regression predict the variables single independent variable

multiple linear regression

predict the variable using multiple variables

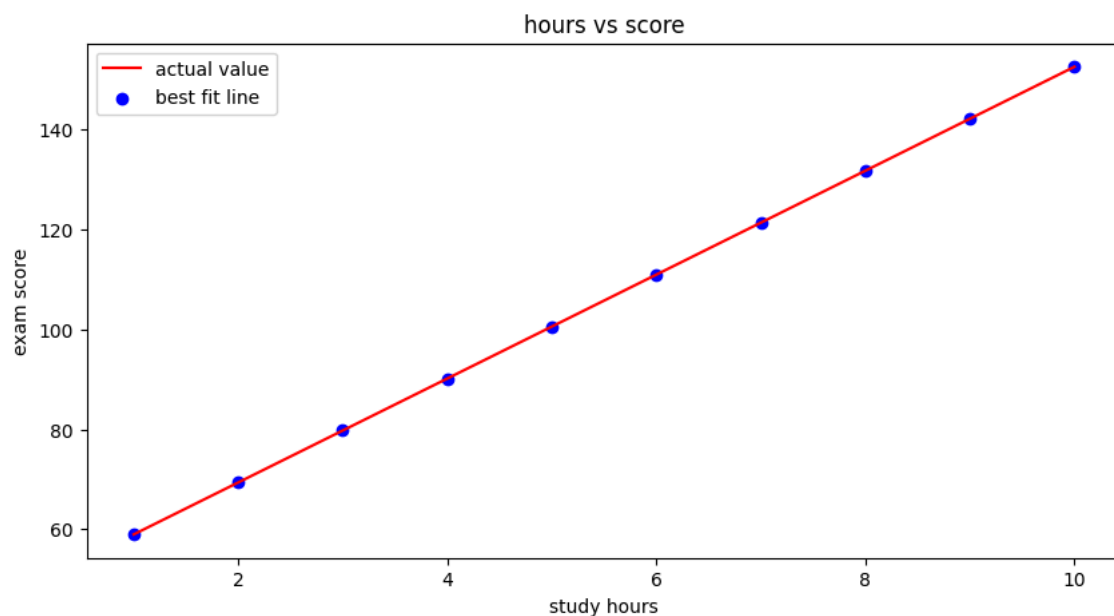
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

```
hours=np.array([1,2,3,4,5,6,7,8,9,10])
score=np.array([60,70,80,90,100,110,120,130,140,150])
```

```
predict_score=10.41*hours+48.52
print("best fit line equation: ")
print("predict_score=10*5+50")
print("predict_score=",predict_score)
```

```
best fit line equation:
predict_score=10*5+50
predict_score= [ 58.93  69.34  79.75  90.16 100.57 110.98 121.39 131.8  142.21 152.62]
```

```
predict_score=10.41*hours+48.52
plt.figure(figsize=(10,5))
plt.plot(hours,predict_score,label="actual value",color='red')
plt.scatter(hours,predict_score,label="best fit line",color='blue')
plt.xlabel("study hours")
plt.ylabel("exam score")
plt.title("hours vs score")
plt.legend()
plt.show()
```



```

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.linear_model import LinearRegression
x=np.array([1,2,3,4,5],dtype=np.float64)
y=np.array([60,70,80,90,100],dtype=np.float64)
m=0
c=0
learning_rate=0.01
epoch=1000
n=len(x)
for i in range(epoch):
    y_pred=m*x+c
    error=y-y_pred
    mse=np.mean(error**2)
    dm=-(2/n)*np.sum(x*error)
    dc=-(2/n)*np.sum(error)
    m=m-learning_rate*dm
    c=c-learning_rate*dc
    if i%100==0:
        print(f'epoch {i}')
        print(f'mse = {mse:.2f}')
        print(f'm= {m:.2f}, c = {c:.2f}')
        print("-----")
print("\n final best fit line:")
print(f'y={m:.2f}x+{c:.2f}')

```

```

epoch 0
mse =6600.00
m= 5.20, c =1.60
-----
epoch 100
mse =177.90
m= 18.63, c =18.84
-----
epoch 200
mse =90.37
m= 16.15, c =27.79
-----
epoch 300
mse =45.90
m= 14.38, c =34.17
-----
epoch 400
mse =23.32
m= 13.12, c =38.72
-----
epoch 500
mse =11.84
m= 12.23, c =41.96
-----
epoch 600
mse =6.02
m= 11.59, c =44.27
-----
epoch 700
mse =3.06
m= 11.13, c =45.92
-----
epoch 800
mse =1.55
m= 10.81, c =47.09
-----
epoch 900
mse =0.79
m= 10.57, c =47.93
-----

final best fit line:
y=10.41x+48.52

```

Afternoon session

```

from sklearn.model_selection import train_test_split
import numpy as np

experience = np.array([1,2,3,4,5,6,7,8,9,10,11,12,13,14,15])
salary = np.array([35,38,42,48,55,60,65,70,76,82,87,92,96,100,105])
x=experience.reshape(-1,1)
y=salary
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
print(f'training sample: {len(x_train)}')
print(f'testing sample: {len(x_test)}')

```

```
training sample: 12
testing sample: 3
```

Start coding or [generate](#) with AI.

```
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
print(f'slope (coefficient):{model.coef_[0]:.2f}')
print(f'intercept:{model.intercept_:.2f}')
```

```
slope (coefficient):5.22
intercept:27.94
```

```
y_pred=model.predict(x_test)
```

```
predicted salary: [80.17937574 90.62741999 33.16317661]
```

```
for actual ,predicted in zip(y_test,y_pred):
    print(f"Actual: {actual:>5} | Predicted: {predicted:>7.1f} | "
          f"Error: {predicted - actual:+.1f}")
```

```
Actual:    82 | Predicted:    80.2 | Error: -1.8
Actual:    92 | Predicted:    90.6 | Error: -1.4
Actual:    35 | Predicted:    33.2 | Error: -1.8
```

1.

Score (Coefficient of Determination) The score represents the proportion of the variance in the target variable that is predictable from the input features

2. MAE (Mean Absolute Error) MAE calculates the average of the absolute differences between the actual values and the predicted values

RMSE is the square root of the average of squared differences (errors) between actual and predicted values

```
from sklearn.metrics import r2_score,mean_absolute_error,mean_squared_error
import numpy as np
y_actual = np.array([500, 300, 700, 450, 600])
y_pred = np.array([480, 320, 690, 460, 590])
r2=r2_score(y_actual,y_pred)
mae=mean_absolute_error(y_actual,y_pred)
mse=mean_squared_error(y_actual,y_pred)
rmse=np.sqrt(mean_squared_error(y_actual,y_pred))
print(f'r2 score:{r2:.4f} -> model explain {r2*100:.1f} % of variance')
print(f'mae:{mae:.1f} -> avg error + {mae:.0f} units')
print(f"RMSE : {rmse:.1f} -> penalised avg error {rmse:.0f} units")
```

```
r2 score:0.9880 -> model explain 98.8 % of variance
mae:14.0 -> avg error + 14 units
RMSE : 14.8 -> penalised avg error 15 units
```

Start coding or [generate](#) with AI.

salary= experience + educaion + age

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score,mean_absolute_error,mean_squared_error
```

```
np.random.seed(42)
n=100
data=pd.DataFrame({
    'experience':np.random.randint(0,20,n),
    'education':np.random.randint(12,20,n),
    'age':np.random.randint(22,55,n)
})
```

```
data['salary']=(
    data['experience']*45000 +
    data['education']*2000 +
    data['age']*300 +
    np.random.randint(0,3000,n)+1000
)
```

```
x=data[['experience','education','age']]
y=data['salary']
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
model=LinearRegression()
model.fit(x_train,y_train)
```

▼ LinearRegression ⓘ ?

LinearRegression()

```
for feat ,coef in zip(x_train.columns,model.coef_):
    print(f"{feat :12s}:$ {coef:.0f} per unit increase ")
print(f'base salary(interpret): {model.intercept_:.0f}')
y_pred=model.predict(x_test)
print(f"\n r2 score: {r2_score(y_test,y_pred):.3f}")
print(f"mae score: ${mean_absolute_error(y_test,y_pred):.0f}")
```

```
experience   :$ 45003 per unit increase
education    :$ 2022 per unit increase
age          :$ 301 per unit increase
base salary(interpret): 2,111

r2 score: 1.000
mae score: $546
```

task of the day

Start coding or [generate](#) with AI.

```
np.random.seed(42)
n=200
location=['urban','suburban','rural']
data=pd.DataFrame({
    'area_sqft':np.random.randint(500,4000,n),
    'num_rooms':np.random.randint(1,7,n),
    'location':np.random.choice(location,n),
    'age_years':np.random.randint(0,40,n)
})
```

Start coding or [generate](#) with AI.